

Devoir à rendre

«Pratique de l'optimisation numérique et sciences des données »

Livrables : Il est demandé de livrer sur Arche au plus tard le **vendredi 06 février 2026 à 14h** un compte-rendu au format pdf du TP ci-dessous.

Consignes : Les réponses aux éventuelles questions et les analyses de vos résultats sont à donner directement dans le code sous la forme de commentaires. Vous indiquerez le numéro de la question le cas échéant. Le compte-rendu final devra être généré par la commande **publish** de MATLAB :

```
publish("nom_du_script.m", format="pdf")
```

Si vous créez des fonctions matlab en plus du script principal, les fichiers .m correspondants devront être déposés en plus du compte-rendu.

Le code devra être commenté, et divisé en sections correspondant aux sous-parties.

Indiquez les difficultés que vous avez rencontrées, et vos tentatives pour les surmonter ; signalez vos sources d'information et les outils utilisés. Si vous échangez au sein de la promotion pour avancer dans la résolution d'un problème, il conviendra de citer la ou les personnes qui vous auront permis de progresser.

L'objectif de ce devoir est de mettre en oeuvre un algorithme de classification SVM sur le jeu de données USPS, qui contient des images de chiffres écrits à la main (de 0 à 9).

1 Jeu de données données USPS

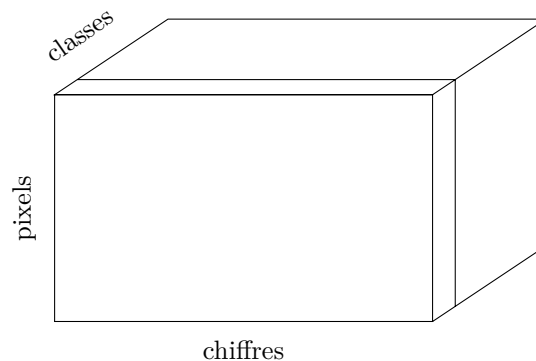
Cette partie s'intéresse à la prise en main et au pré-traitement des données.

1.1 Construction du jeu de données

Le jeu de données se trouve dans le fichier `usps_all.mat` et est représenté par un tenseur `data` d'ordre 3. Les dimensions du tenseur sont $256 \times 1100 \times 10$.

```
load("usps_all.mat"); % charger 'data'
data = double(data);
```

Chaque colonne `data(:,j,k)` est la vectorisation d'une image, pour laquelle l'indice `k` correspond à une classe (de 0 à 9) et l'indice `j` correspond à un échantillon (ici, un exemple du chiffre). Ainsi, la première dimension correspond aux pixels, la deuxième aux chiffres, et la troisième aux classes, comme montré dans la figure ci-dessous. L'ordre des classes est '1', '2', '3', ..., '0'.



1. Afficher quelques exemples des images d'entraînement, dans les classes 7 et 9 par exemple, à l'aide de `imagesc`.
2. En apprentissage automatique, il est d'usage de séparer le jeu de données en deux parties : une partie servant à l'entraînement du modèle, l'autre à sa validation. Diviser le jeu de données en deux tenseurs `T` et `V`, en sélectionnant des sous-tenseurs de `data` selon la deuxième dimension (chiffres). Les indices `ind_train` et `ind_valid` correspondant aux échantillons à extraire dans chaque sous-tenseurs sont donnés dans le fichier `usps_partition.mat`, qui se charge comme avant avec la commande `load`.

1.2 Analyse en composantes principales (ACP)

3. Pour les données d'entraînement `T`, calculer le dépliement du tenseur `T` selon le mode 1 (une matrice avec 256 lignes) à l'aide de la commande `reshape`, et trouver la matrice `U` des dix premiers vecteurs singuliers gauches du résultat.

-
4. Replier chaque vecteur singulier en une matrice 16×16 à l'aide de **reshape** et les afficher avec **imagesc** dans une même figure.

Nous allons entraîner un classifieur SVM sur deux des 10 classes (les chiffres 7 et 9).

5. Sélectionner dans **T** les données des deux classes 7 et 9, et convertir ces données labellisées en :
1) une matrice **Xt(chiffre,pixel)** de 256 colonnes, et 2) un vecteur **yt(classe)** de labels.
6. Décomposer de même les données de validation en matrice **Xv** et vecteur **yv**.
7. Pour les données d'entraînement, appliquer la réduction de dimension pour passer de l'espace de dimension 256 à l'espace de dimension 10 ($\widehat{\mathbf{X}} = \mathbf{X}\mathbf{U}$).

2 Classification SVM

Le classifieur SVM de Matlab est **fitcsvm**. La commande

```
svm = fitcsvm(X,y);
```

renvoie un classifieur SVM entraîné sur les données contenues dans **X**, avec labels **y**. Cette fonction prend par ailleurs plusieurs options (consulter son aide), notamment :

- 'BoxConstraint' : si cette option est fixée à **inf**, le classifieur utilise une marge dure. Pour toute autre valeur strictement positive, la marge est souple (la valeur par défaut est 1)
- 'KernelFunction' : le noyau utilisé le cas échéant
- 'KernelScale' : dans le cas d'un noyau gaussien, ce paramètre correspond à l'écart-type
- 'Verbose' : affiche ou non les informations liées au classifieur pendant l'optimisation

2.1 Classification

8. Entraîner un classifieur SVM à marge souple et à noyau gaussien sur les données projetées. Attention : on utilisera l'option **..., 'Standardize', true, ...** pour normaliser les données.

Une fois le classifieur entraîné, les valeurs de labels prédites à partir des données d'entraînement (après ACP) se calculent avec la commande **predict**, qui peut également renvoyer un score pour chaque label, correspondant à la probabilité que ce label soit le bon.

```
[labels,scores] = predict(svm,X_ACP);
```

9. Calculer le taux d'erreur de classification sur les données d'entraînement, c'est-à-dire le ratio entre le nombre de mauvaises classifications et le nombre total d'échantillons.
10. Réduire la dimension des données de validation avec la même matrice **U** qui a été utilisée pour les données d'entraînement. Calculer les prédictions et le taux d'erreurs de classification sur le jeu de validation.

2.2 Réglages des hyperparamètres

11. Répéter les codes précédents pour différentes valeurs du paramètre 'KernelScale' dans l'intervalle [0.5;20]. Tracer les courbes des erreurs d'apprentissage et de validation correspondant. Quelle est une bonne valeur de ce paramètre ?

```
sigma_list = 0.5:0.5:20;
% Insérer votre code ici
for i=1:length(sigma_list)
    % Insérer votre code ici
end
% Insérer votre code ici (plot)
```

12. Répéter la procédure pour deux autres classes (chiffres 6 et 8). Est-il plus difficile de séparer ces classes ? La courbe d'erreur de validation est-elle différente de la précédente ?
13. Une façon courante et pratique de régler les hyperparamètres (comme 'KernelScale') est d'utiliser la méthode de validation croisée. Cette méthode n'utilise que les données d'apprentissage, en les sous-divisant d'une manière astucieuse en sous-ensembles d'entraînement et de validation. On peut optimiser le paramètre 'KernelScale' de la façon suivante :

```
svm_opt = fitcsvm(X,y,'Verbose',1, ...
    'Standardize',true, ...
    'KernelFunction','gaussian', ...
    'KernelScale',1, ...
    'OptimizeHyperparameters','KernelScale');
sigma_opt = s_opt.KernelParameters.Scale;
```

Comparer les résultats (paramètre optimal, taux d'erreur) avec l'erreur de validation obtenue à l'étape précédente.

2.3 Pré-traitements additionnels

14. Prenez deux photos (noir et blanc, carré) avec votre smartphone, représentant les deux classes (6 et 8), ou utilisez les photos fournies dans les fichiers du tp. Affichez-les dans Matlab (NB : une image peut-être lue depuis un fichier avec la commande `imread`). Classifiez-les après les étapes de pré-traitement suivantes :
 - Si l'image est en couleurs, conversion en niveaux de gris (en prenant par exemple la moyenne des canaux RGB)
 - Recadrage (si nécessaire)
 - Négatif (blanc sur fond noir)
 - Correction du contraste – il faut normaliser les données de telle façon que les valeurs minimales et maximales deviennent 0 et 255 : $f_{i,j} = \frac{f_{i,j} - f_{\min}}{f_{\max} - f_{\min}} \cdot 255$
 - Redimensionnement à l'aide de la fonction `imresize`.

3 Classification multi-classes

Il y a deux stratégies principales pour entraîner un classifieur multi-classe (avec M classes) à partir des outils de classification bi-classes que nous avons vues :

One-vs-one : on entraîne des classifieurs SVM bi-classes pour chacune des paires de classes. Par exemple : 1 vs. 2, 1 vs. 3, ..., 1 vs. 0, 2 vs. 3, ..., 2 vs. 0, ... etc. Pour trouver une prédiction, on évalue ensuite tous les classifieurs sur un nouvel échantillon, et on choisit la classe qui ressort le plus souvent. Par exemple :

- '1' vs. '2' : '1'
- '1' vs. '3' : '1'
- '2' vs. '3' : '3'

⇒ la classe '1' est choisie.

One-vs-all : On entraîne M classifieurs pour la classification binaire :

- '1' vs. '2','3',... '0'
- '2' vs. '1','3',... '0'
- ⋮
- '0' vs. '1','2',... '9'

et pour un nouvel échantillon, on choisit la classe ayant obtenu le meilleur score (voir section 2.1).

15. Implémenter une des deux stratégies pour le jeu de données USPS. Calculer la matrice de confusion (commande `confusionmat`) et le taux d'erreur.